

Class: CCCS 315-784

Student Name : Cynthia Maroupas

Student ID: 261119382

Part 1: Priority Queues Analysis

1. Big-O analysis of each algorithm:

Heap Sort has a time complexity of $O(n \log n)$ in both average and worst cases, as building the heap takes $O(n)$ time, and each removal operation takes $O(\log n)$ time. With n removals, the total complexity is $O(n \log n)$.

Insertion Sort has a time complexity of $O(n^2)$ in the worst and average cases, as it involves comparing and swapping elements in a nested loop. However, in the best case (when the input array is already sorted), the time complexity is $O(n)$ since it only needs one pass through the array.

Selection Sort has a time complexity of $O(n^2)$ for the best, worst, and average cases. It performs $n-1$ comparisons for the first element, $n-2$ for the second, and so on, resulting in a total of $(n-1) + (n-2) + \dots + 1 = n(n-1)/2$ comparisons, making the time complexity $O(n^2)$.

2. Which sorting algorithm is faster?

Based on the Big-O analysis, Heap Sort is generally faster than Insertion Sort and Selection Sort, as it has a more efficient time complexity of $O(n \log n)$. Heap Sort has the advantage of being able to maintain a balanced structure while sorting, which contributes to a better performance. On the other hand, Insertion Sort and Selection Sort have quadratic time complexities, $O(n^2)$, making them less efficient, particularly for bigger input sizes.

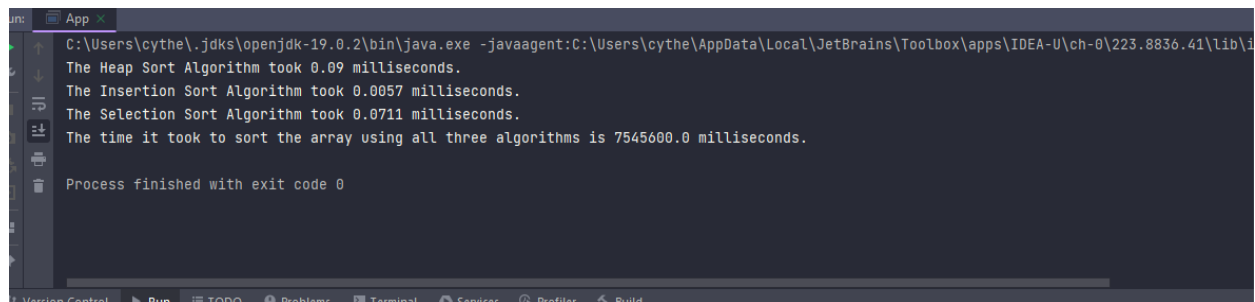
3. Do you think if we use array or LinkedList, we could speed the program up?

Using an array or LinkedList has different effects on the algorithm performance. Arrays benefit algorithms like Heap Sort and Selection Sort due to constant-time access. Linked List, with constant-time insertions and deletions, can be more suitable for algorithms like Insertion Sort. However, arrays are usually more cache-friendly and have better overall performance.

4. Does the array initial order have any effect on the speed?

The initial order of the array does affect the speed of some sorting algorithms. As an example, Insertion Sort has a best-case time complexity of $O(n)$ when the array is already sorted, but $O(n^2)$ in the average and worst cases. Heap Sort and Selection Sort have the same time complexity, regardless of the initial order of the array. However, the actual execution time may vary slightly depending on the input data, even if the time complexity remains the same.

Screenshot of the console for part 1 Priority Queues:



```
un: App
C:\Users\cythe\.jdk\openjdk-19.0.2\bin\java.exe -javaagent:C:\Users\cythe\AppData\Local\JetBrains\Toolbox\apps\IDEA-U\ch-0\223.8836.41\lib\i
The Heap Sort Algorithm took 0.09 milliseconds.
The Insertion Sort Algorithm took 0.0057 milliseconds.
The Selection Sort Algorithm took 0.0711 milliseconds.
The time it took to sort the array using all three algorithms is 7545600.0 milliseconds.

Process finished with exit code 0
```

References :

Book: Data Structures and Algorithms in Java – Michael T. Goodrich

<https://www.geeksforgeeks.org/priority-queue-set-1-introduction/>

<https://www.geeksforgeeks.org/difference-between-insertion-sort-and-selection-sort/>

<https://www.geeksforgeeks.org/insertion-sort/?ref=lbp>

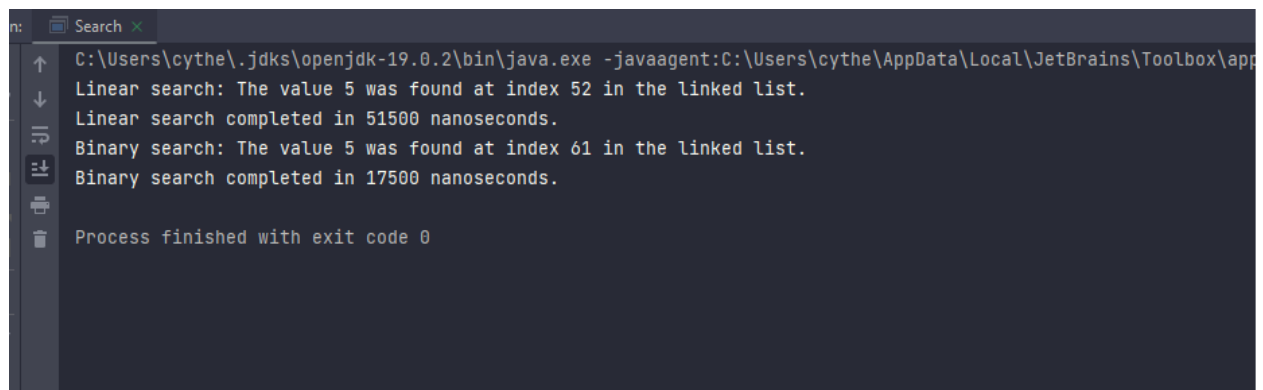
Part 2: Search

Big-O analysis of the running time of both methods:

Linear search: In the worst-case scenario, the target value is either not in the list or located at the final index. In such cases, the algorithm must go through the entire list. With a list containing n elements, the worst-case time complexity for linear search is $O(n)$.

Binary search: In the worst-case scenario, the target value is either not in the list, or the search interval keeps getting smaller by half until it finally reaches the target value. Given a list with n elements and the interval being reduced by half at each step, the worst-case time complexity for binary search is $O(\log n)$.

Screenshot of the console for part 2 Search:

A screenshot of a console window titled "Search" with a search bar. The console output shows the execution of a Java program. The first line is the command: `C:\Users\cythe\.jdk\openjdk-19.0.2\bin\java.exe -javaagent:C:\Users\cythe\AppData\Local\JetBrains\Toolbox\app`. The output then shows: "Linear search: The value 5 was found at index 52 in the linked list.", "Linear search completed in 51500 nanoseconds.", "Binary search: The value 5 was found at index 61 in the linked list.", "Binary search completed in 17500 nanoseconds.", and finally "Process finished with exit code 0".

```
n: Search x
C:\Users\cythe\.jdk\openjdk-19.0.2\bin\java.exe -javaagent:C:\Users\cythe\AppData\Local\JetBrains\Toolbox\app
Linear search: The value 5 was found at index 52 in the linked list.
Linear search completed in 51500 nanoseconds.
Binary search: The value 5 was found at index 61 in the linked list.
Binary search completed in 17500 nanoseconds.

Process finished with exit code 0
```